Forecasting Bike Rental Demand ### Project Report — UE24CS352A Machine Learning Mini-Project (Problem No. 87) **Dabbugunta Venya Anand** (PES1UG24AM074) **Dhruv Talavat** (PES1UG24AM087) Reference paper — Du, He & Zhechev, *Forecasting Bike Rental Demand* (Kaggle bike-sharing competition)

1. Introduction and Problem Statement

Bike-sharing systems live or die on how well the operator can guess tomorrow's demand. Put too few bikes out before the morning rush and you lose rides (and annoy commuters); leave docks overflowing at midnight and you have wasted rebalancing effort and blocked parking. Both are real money problems for operators like Capital Bikeshare in Washington D.C., which is the system whose data we work with here.

So the task we were given (Problem 87) is: **given the hour, day and weather, predict how many bikes will be rented in that hour.** Formally this is supervised regression on hourly counts. We use two years of hourly records (2011–2012), with calendar features (hour, weekday, month, season, holiday / working-day flags) and weather features (feels-like temperature, humidity, wind speed, weather situation).

Because our reference paper competed on Kaggle with this exact dataset, we adopted their headline metric, the **Root Mean Squared Logarithmic Error (RMSLE)**, alongside RMSE / MAE / R^2. RMSLE compares errors on a log scale, which suits count data well — being off by 10 at 3 AM is less serious than being off by 10 at 6 PM, and RMSLE reflects that instead of treating every hour equally.

At the review session we demo a small Streamlit app: the user picks a scenario (e.g. a Monday 6 PM commute, cloudy, 27 °C) and the trained model returns a forecast for that hour.

2. Dataset

Source: UCI Machine Learning Repository — Bike Sharing Dataset, hourly file (`hour.csv`).

Property	Value
Records	17,379 hourly rows
Period	2011-01-01 to 2012-12-31
Target	`cnt` = `casual` + `registered` (rentals in that hour)
Target range	1 to 977 rentals/h, mean ≈ 189, right-skewed (skew ≈ 1.28)
Missing / duplicates	None found (checked all rows; no repeated `(date, hour)` pairs)
Columns used	calendar: `season`, `yr`, `mnth`, `hr`, `weekday`, `holiday`, `workingday`; weather: `weathersit`, `temp`, `atemp`, `hum`, `windspeed`

Two dataset details mattered a lot during preprocessing:

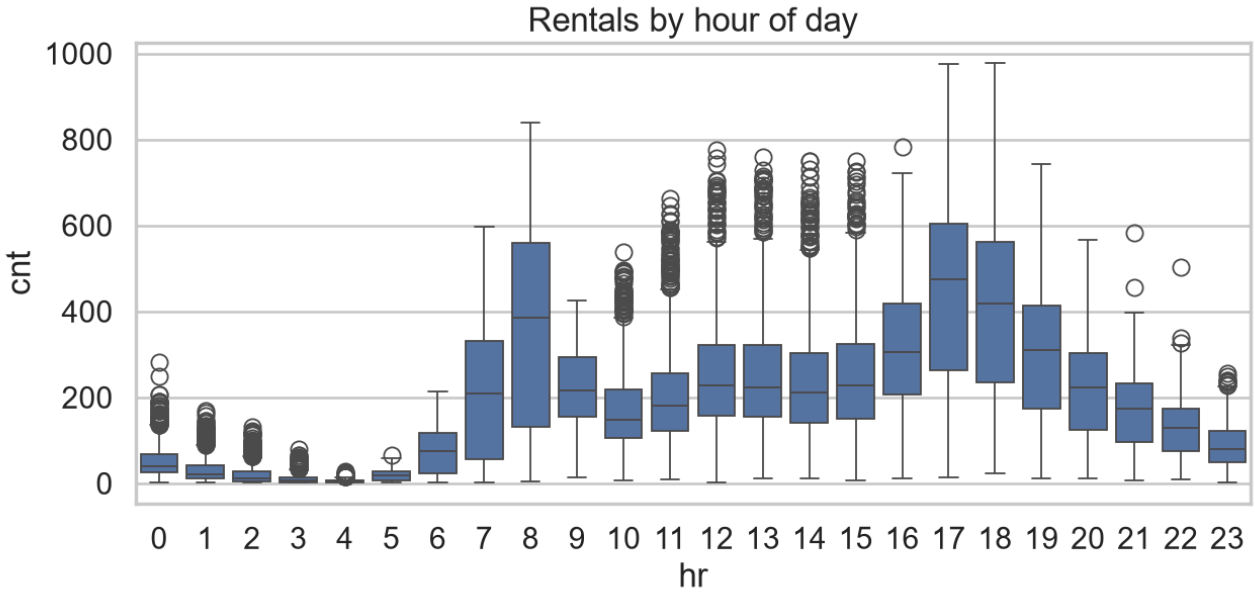
- **User-type columns are leakage.** `casual + registered = cnt` exactly, so they can never go into a feature set. The reference paper's §5.5 tried modelling the two user types separately and found it *worse* than predicting the total, which matched what we saw in the EDA, so we exclude them everywhere.
- **The weekday coding is Sunday=0 … Saturday=6.** We verified this empirically (every Saturday in the data has `weekday=6`, and medians per calendar day confirm the mapping). This bit us once: a `weekday >= 5` weekend check actually flags Friday+Saturday. We fixed it to `weekday in (0, 6)` — a small thing, but it silently corrupts a weekend feature if missed.
- **`temp` and `atemp` are ~0.99 correlated**, so keeping both buys nothing. Following the paper's featurization experiment we keep `atemp` (feels-like temperature is what a rider actually reacts to) and

drop raw `temp`.

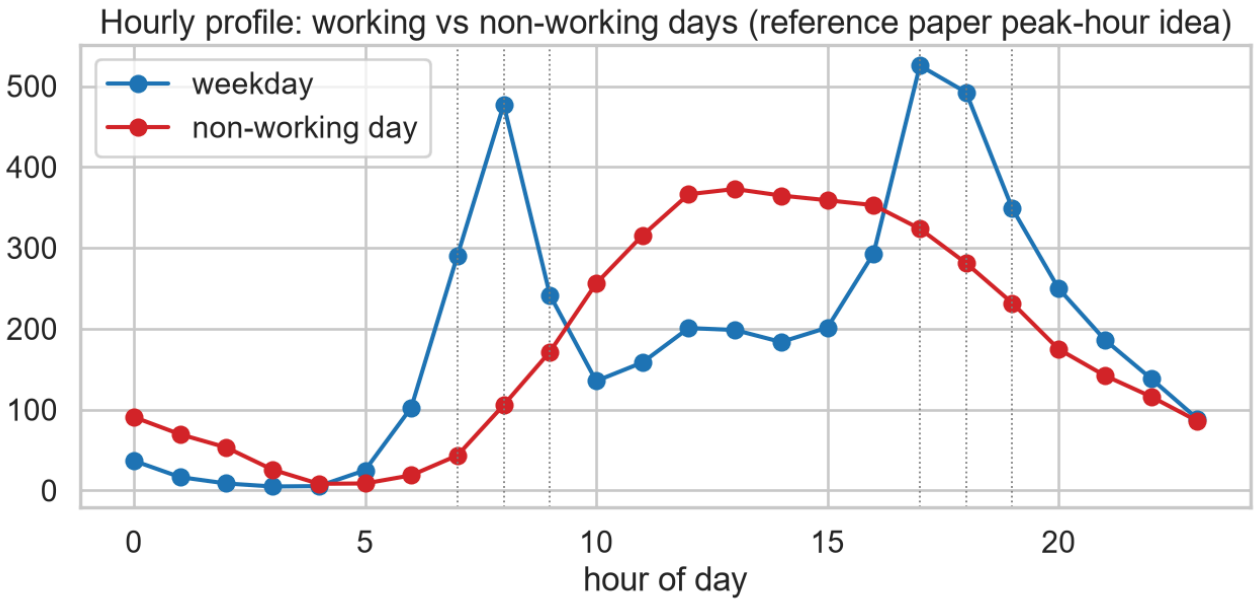
3. Exploratory Data Analysis

All figures in this section are in `reports/` and are produced by `src/data_understanding.py`, so they can be regenerated with one command.

Hourly demand is strongly bimodal on working days.



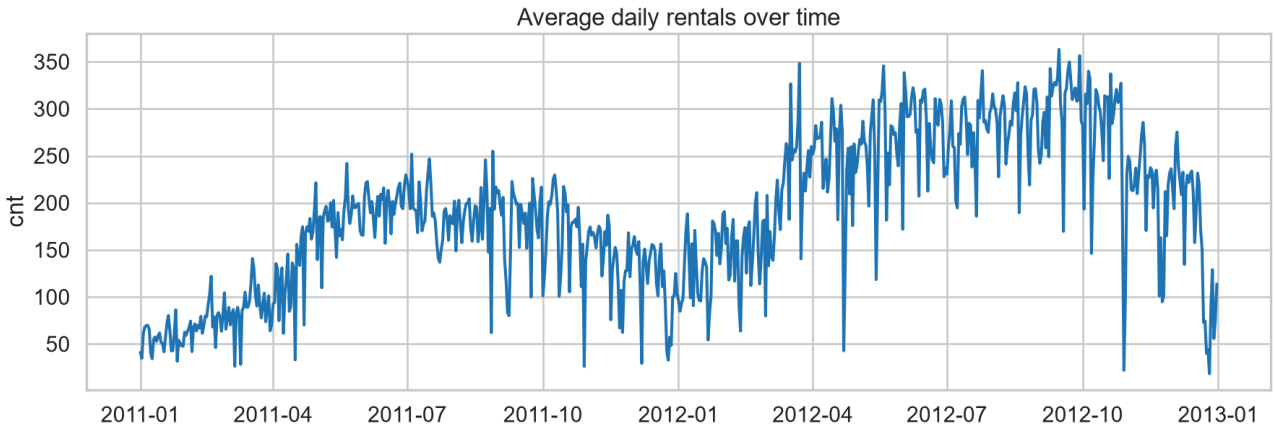
Working days show two clear peaks — a morning spike around 8 AM and a larger evening one around 5–6 PM. Non-working days are completely different: a single broad hump from late morning to early evening, no commute spikes.

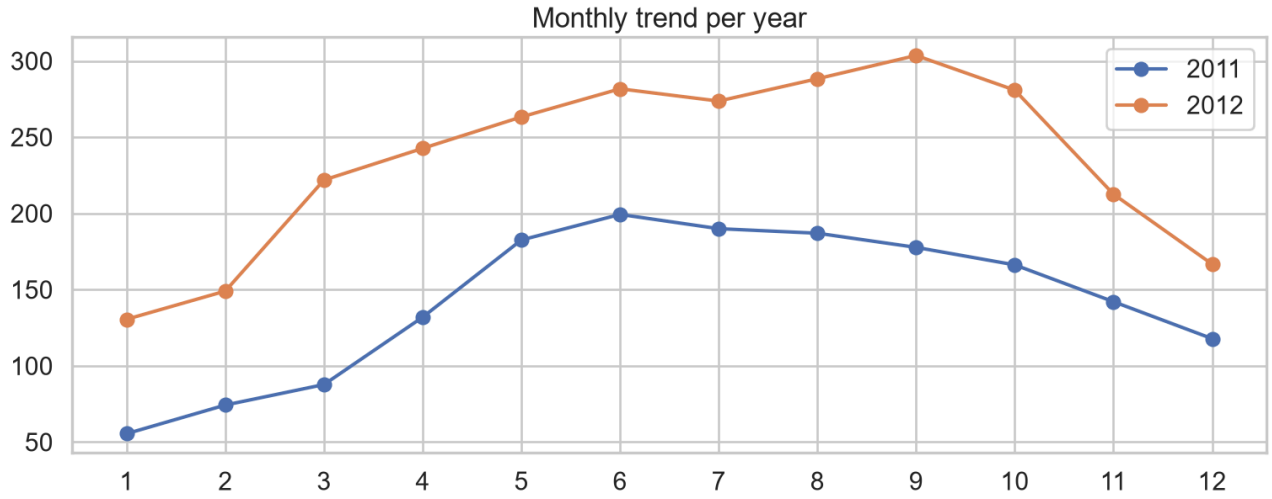


The dashed lines in this figure mark the cut-offs we chose for a `peak_time` feature: weekday peaks 7–9 AM and 5–7 PM, weekend "peak" 10 AM–6 PM. This is the one EDA finding that fed directly into the feature

set (and it matches the peak-hour observation in the reference paper).

Demand grows year-over-year and is strongly seasonal.





2012 is visibly higher than 2011 for every month (the system grew), and both years peak in summer (Jun–Sep) and bottom out in winter. We keep the `yr` flag so the model can absorb that growth rather than pretending year 2 will look like year 1.

Weather matters, but less than time. Correlations with `cnt`: feels-like temperature ≈ +0.39, humidity ≈ −0.32, wind speed only ≈ −0.06. Demand drops sharply as the weather goes Clear → Cloudy → Wet, and weather situation 4 (severe) has almost no observations, so any prediction in that regime is extrapolation, not learning.

4. Preprocessing and Feature Engineering

Implemented in `src/preprocess_and_baseline.py`; ideas follow the paper's §2.2 featurization experiments.

1. **Scaling** — `StandardScaler` on the numeric block for linear models; trees get raw values (passthrough), since scaling is irrelevant to them.
2. **Encoding** — `OneHotEncoder(handle_unknown="ignore")` for the categorical block, the same "convert categoricals to indicators" practice the paper used for its linear models. Unseen categories map to all-zeros instead of crashing at inference.

3. **Cyclic time encodings** — hour, weekday and month are every-bit-periodic, so we add `sin/cos` pairs. The point is distance: 23:00 ends up numerically next to 00:00, and December–January stops looking like an unrelated pair of categories.
4. **Temperature buckets** — the paper discretized temperature to help linear models cope with its non-linear effect, so we bin `atemp` into four buckets (cold/mild/warm/hot). One subtlety worth recording: the bin edges are **fitted on the training split only** and then reused unchanged for the test split and the demo app. Our first version binned train and test independently, which means the buckets silently disagree between the two; the fix keeps the discretization consistent and leakage-free.
5. **Peak time buckets** — the three-way feature justified by Figure 3 above (`weekday_peak` / `weekend_peak` / off-peak), plus a simple `is_rush_hour` flag.

What we deliberately excluded: `cnt`, `casual`, `registered` (leakage), `instant` (row index), and raw `dteday` and `temp`. We also *kept* the `holiday` flag even though the paper dropped theirs — in the full UCI data holidays are ~1.9 % of rows and mid-week holidays behave like weekends, so the flag carries signal rather than noise.

5. Methodology — Train/Test Split and Metric

The Kaggle split used in the reference paper slices the month into "first 20 days train / last ~10 days test", which leaks future information across the boundary (a test day's neighbours may be in training). The paper itself lists proper time-ordered evaluation as future work, so we do exactly that:

- **Train:** 2011-01-01 → 2012-08-07 (13,915 rows, ≈ 80 %)
- **Test:** 2012-08-08 → 2012-12-31 (3,464 rows, ≈ 20 %)

The model never sees data from after its test window. Model selection inside the training period used `TimeSeriesSplit(4)` with RMSE as the grid-search score, and all reporting is on the untouched test period. Primary metric: RMSLE; RMSE / MAE / R^2 shown alongside for comparison with the paper.

6. Implementation Overview

```
data/hour.csv                    UCI Bike Sharing, hourly (data/ is gitignored;
                                 scripts/download_data.py fetches it)
src/data_understanding.py        schema/quality checks, 12 EDA figures -> reports/
src/preprocess_and_baseline.py   features, temporal split, LR + RF baselines
src/advanced_models.py           GBM, RBF-SVR, GBM+RF blend -> advanced_metrics.json
app/demo_app.py                  Streamlit demo: scenario inputs -> prediction
models/                          fitted pipeline + metrics/diagnostics JSON
reports/                         all EDA and diagnostic figures
scripts/                         slide generator, md -> PDF renderer, data fetcher
```

The whole pipeline reproduces in three commands from a fresh clone (`python src/data_understanding.py`, `python src/preprocess_and_baseline.py`, `python src/advanced_models.py`, then `streamlit run app/demo_app.py` — exact setup steps, including the one-time dataset download, are in the README).

The demo app applies the **same featurization as training** — including the train-fitted temperature-bucket edges — so a scenario picked in the sidebar is transformed exactly the way the training rows were. If you type "Monday, 6 PM, cloudy, feels like 27 °C" you get the model's best estimate for that hour in real time.

7. Models and Results

Two baselines were tuned first, then the paper's advanced-model ideas were run on identical features and split.

- **Linear Regression** — scaled numerics + one-hot categoricals, the paper's §3.1 baseline.
- **Random Forest** — grid over `n_estimators` (200/400), `max_depth` (12/18/None), `min_samples_leaf` (1/3/5), 4-fold time-series CV. Best: `max_depth=18, n_estimators=400, min_samples_leaf=1`.
- **Gradient Boosting** — 400 trees, depth 5, learning rate 0.1, subsample 0.8 (paper's GBM).
- **RBF-SVR** — C=10, ε=0.1 (paper's SVR; by far the slowest to train).
- **GBM + RF blend** — a plain 50/50 average of the two tree ensembles' predictions, our cheapest model-combination attempt (the paper's stacking idea in miniature).

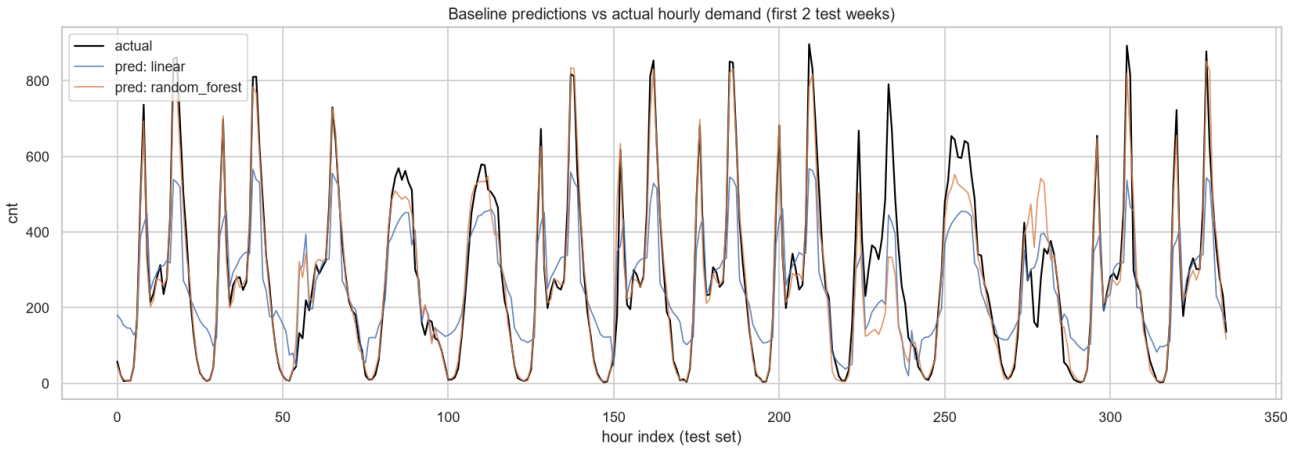
Test-set results (the reviewed evaluation: one model, one temporal test period):

Model	RMSLE	RMSE	MAE	R^2
Linear Regression	1.045	121.1	91.0	0.698
Random Forest	0.410	80.0	52.8	0.868
Gradient Boosting	0.502	**65.3**	42.9	**0.912**
RBF-SVR	0.558	106.1	68.5	0.768
GBM + RF blend	**0.403**	70.9	46.7	0.896

Two readable patterns:

- **Trees crush the linear model.** RMSLE 0.41–0.50 vs 1.045, exactly the paper's central finding (their best tree CV-RMSLEs 0.46–0.50 vs linear ~1.04). The hourly demand curve is a train of sharp commuter spikes and overnight troughs; no linear combination of these features traces that shape.
- **No single winner.** By RMSLE our best is the blend (0.403); by RMSE/R^2 it is pure GBM. This lines up with the paper's own reasoning for ensemble methods — different models are right about different hours, and mixing them helps most when one of them underestimates the tail.

Checking the predictions against reality over the first two test weeks (Random Forest, orange, vs the actual series in black):



The RF tracks the daily shape almost peak-for-peak; the visible misses are mostly on days with unusual weather, and systematically *under* forecast the tallest evening peaks. Residual plots

(`reports/14_baseline_residuals.png`) tell the same story: fairly balanced at low counts, mildly biased high around the peaks. Against the reference paper, our RF (0.410) lands close to their best models (CTree 0.460, RF 0.503) even though their CV numbers are not directly comparable to a stricter time-ordered split — the "trees win" pattern is the same.

8. Demo

The Streamlit app (`app/demo_app.py`) is deliberately minimal: a sidebar for the scenario (season, date pieces, weather condition, holiday/working-day flags), sliders for weather values, and an instant forecast. Internally it mirrors the training featurization byte-for-byte:

- the **train-fitted** temperature-bucket edges are loaded from `models/temp_bucket_edges.json`, not recomputed at demo time;
- feels-like temperature is handled in the dataset's normalized space (the raw UCI values are `value/50`), so a slider value in °C is converted once and then compared against normalized bucket edges;
- the weekday map follows the dataset's Sunday=0 … Saturday=6 convention (confirmed against `dteday` during EDA), which matters because two earlier versions of this mapping silently produced wrong weekend flags — a good lesson in reading dataset documentation carefully instead of assuming Monday=0.

Run: `.venv/Scripts/streamlit run app/demo_app.py`.

9. Conclusions

Working from problem to result:

1. **The problem is mostly a problem of encoding time well.** Of everything we tried, the cyclic hour encodings and the peak-bucket feature carried by far the most signal; weather adds a real but secondary correction on top. The paper's variable-importance study says the same thing.
2. **Baseline quality was driven by following the paper's featurization experiments**, not by exotic modelling: RMSLE 1.045 (linear) → 0.410 (tuned RF) with only standard components.
3. **Ensembling was the last cheap win:** the GBM+RF blend edges out everything on RMSLE (0.403) at essentially zero extra cost, while pure GBM remains best on RMSE/R^2. We would nominate the blend as the delivery model.
4. **Methodology hygiene mattered more than we expected.** The leakage audit (excluding `casual`/`registered`), the train-only temperature-bucket edges, and the temporal split all changed reported numbers or made them trustworthy at all. The single biggest debugging session came from the weekday-code convention, and we would add a data-documentation check to every future project's step zero.
5. **Limitations we would flag honestly:** severe-weather regime is effectively unseen (predictions there are guesses); two years of history with built-in growth limit any long-horizon extrapolation; and RFs cannot predict above their training maximum, which the GBM compensates partially. The paper's §7.3 suggestion of a true time-series model trained only on recent history is the natural next step, as is giving the demo a severity regime check rather than letting it predict severe-weather hours with unearned confidence.

10. Work Division

Person	Responsibility
Dhruv Talavat (PES1UG24AM087)	Dataset understanding & EDA, preprocessing/feature engineering, baseline models (LR, RF), temporal-split methodology, demo app input side, Person A write-up & slides
Dabbugunta Venya Anand (PES1UG24AM074)	Advanced models (GBM, SVR, blend), model comparison & best-model selection, inference pipeline integration, demo prediction side, models/results write-up & slides
Both	README, method alignment with the reference paper, integration & testing, final review / viva preparation

11. Repository

Private GitHub repo shared with faculty: contains all source (`src/`, `app/`, `scripts/`), the dataset-fetch script, all model artefacts and figures (`models/`, `reports/`), and this documentation. Setup is a three-command flow from the README, and every figure and metric in this report is reproducible from that flow.